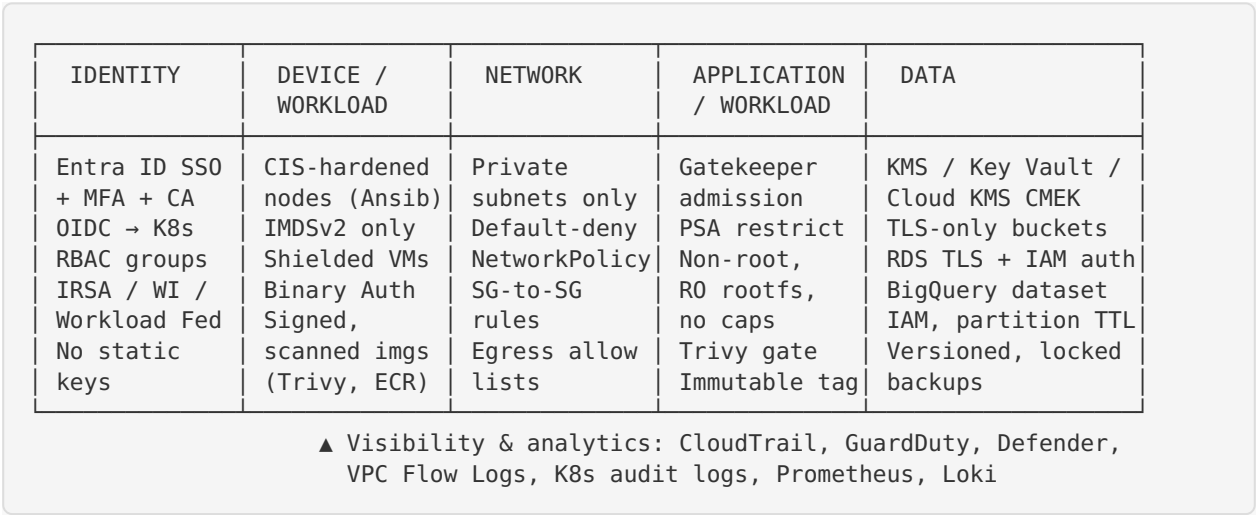
CzarIntel Zero Trust Model

> Zero Trust means **identity is the perimeter**. No network location, IP range, cluster membership or long-lived secret grants access by itself. Every request is authenticated, authorised against least privilege, encrypted in transit, and logged.
>
> This document follows the NIST SP 800-207 pillars and cites the file that enforces each control.

---

1. The five pillars



---

2. Identity: no static credentials anywhere

Actor	Authenticates to	Mechanism	Credential lifetime	Enforced in
Engineers	All consoles, `kubectl`, Grafana, ArgoCD	Entra ID OIDC, MFA, Conditional Access	Token ~1 h	`terraform/modules/azure` (`azure_active_directory_role_` `local_account_disabled = true`), `monitoring/prometheus/values-override.yml` (Grafana `auth.azuread`)
Engineers → K8s	EKS / AKS / GKE API	Entra groups mapped to ClusterRoles	Token ~1 h	`kubernetes/rbac/roles.yml` (`czarintel:platform-admins`, `:sre`, `:developers`, `:auditors`)
api-service → AWS	Secrets Manager, KMS, S3 prefix	**IRSA**: projected SA token → `sts:AssumeRoleWithWebIdentity`, `sub` pinned to `system:serviceaccount:production:api-service`	1 h STS	`terraform/modules/aws/main.tf` `aws_iam_role.api_service`
Jenkins agents → ECR	ECR push (one repository)	**IRSA**, `sub` pinned to `jenkins:jenkins-agent`, `max_session_duration = 3600`	1 h STS	`aws_iam_role.jenkins_agent`, `ci-cd/Jenkinsfile` (no AWS credentials binding)
Workloads → Azure Key Vault	Secrets	**Workload Identity Federation**: federated credential on an Entra app, trusted issuer = AKS OIDC	~1 h	`azuread_application_federated_identity_credential.workload`

Actor	Authenticates to	Mechanism	Credential lifetime	Enforced in
api-service → GCP	GCS, BigQuery dataset	**GKE Workload Identity**: KSA → GSA via authoritative `roles/iam.workloadIdentityUser` binding	1 h	`google_service_account_iam_binding.api_service_workload_identity`, `google_bigquery_dataset_iam_member.api_service_writer`
GitHub Actions → AWS	Read-only compliance and cost APIs	**GitHub OIDC** → `aws-actions/configure-aws-credentials` role assumption	1 h	`.github/workflows/platform-quality.yml` (`permissions: id-token: write`)
App → RDS	MySQL	TLS required (`require_secure_transport = ON`); IAM DB auth enabled. The master secret is RDS-managed and rotated in Secrets Manager.	15 min IAM token	`aws_db_parameter_group.mysql`, `aws_db_instance.mysql`

Remaining long-lived secrets, and why they are acceptable:
- `gitops-deploy-key`: a deploy key scoped to one repository, write-only to that repo.
- `slack-webhook-url`: can only post to one channel.

Both are stored in Jenkins' encrypted credential store, and both are candidates for GitHub App tokens and Slack OAuth in roadmap week 11.

---

3. Network: default-deny, explicit allow

3.1 Cloud network layer

- **AWS:**

- EKS nodes and RDS run only in private subnets. NAT provides egress, and there is no public IP on nodes.
- The **default security group is stripped** (CIS AWS 5.4).
- Security groups reference each other, never CIDRs: `rds_from_nodes` allows 3306 only from the node SG.
- EKS API public access is disabled unless `public_access_cidrs` is set.
- VPC Flow Logs are on for ALL traffic, sent to KMS-encrypted CloudWatch.
- **Azure:**
- The AKS subnet has an NSG.
- Azure CNI Overlay runs with **Calico** network policy.
- The API server is restricted to `api_authorized_ranges`.
- **GCP:**
- GKE is private (private nodes, and a private endpoint when no authorised CIDRs are provided).
- **Dataplane V2** (eBPF) enforces NetworkPolicy.
- Master authorised networks are configured.

3.2 Kubernetes layer: `kubernetes/network-policies/default-deny.yml`

Namespace	Baseline	Explicit allows
`production`	`default-deny-all` (ingress and egress)	DNS egress to kube-dns · ingress-nginx → api-service :8000 · Prometheus scrape · api-service → mysql :3306 (with a matching ingress policy on the mysql side) · api-service HTTPS egress
`staging`	`default-deny-all`	DNS egress · Prometheus scrape
`monitoring`	`default-deny-all`	Intra-namespace traffic · ingress → Grafana · log shipping → Loki · operator admission webhook
`jenkins`	`default-deny-all` (`ci-cd/jenkins/jenkins-deployment.yml`)	Ingress to the controller UI/agent port · controlled egress

`scripts/compliance_scan.py` check **K8S-005** fails the build if any namespace that runs workloads lacks a default-deny policy.

---

4. Workload and admission control

Admission happens in two stages. Both must pass.

1. **Pod Security Admission, `restricted` profile** (`kubernetes/namespaces.yml` , labels `pod-security.kubernetes.io/enforce: restricted`). It rejects root users, privilege escalation, host namespaces, hostPath, missing seccomp profiles and added capabilities.
2. **OPA Gatekeeper 3.17.1** (`kubernetes/opa-gatekeeper/require-labels.yml`), with every constraint set to `enforcementAction: deny` :

Constraint	Effect	Why
`workloads-must-have-ownership-labels`	Deployments, StatefulSets and DaemonSets must carry `app.kubernetes.io/name` , `czarintel.io/owner` and `czarintel.io/cost-center` (`^CZ-[0-9]{4}$`)	Accountability (SOC2 CC1.3), cost allocation
`namespaces-must-declare-environment`	Namespaces need `czarintel.io/environment ∈ {production, staging, shared, dev}`	Environment segregation (PCI 6.4.1)
`approved-registries-only`	Pods may pull only from our ECR, ACR, Artifact Registry and a short list of pinned upstreams	Supply-chain integrity (NIST SI-7, SR-3)
`block-latest-tag`	Rejects `:latest` and untagged images	Reproducibility, change control (NIST CM-2)

Workload-level hardening, enforced by the Helm charts and verified by compliance checks K8S-001..004:
- `runAsNonRoot` , `readOnlyRootFilesystem` , `allowPrivilegeEscalation: false` , `capabilities.drop: [ALL]` , `seccompProfile: RuntimeDefault`
- CPU and memory requests and limits on every container, including initContainers
- `automountServiceAccountToken: false` , except where IRSA is needed

---

5. Supply chain

```
source → flake8/hadolint → pytest → Kaniko (rootless) → Trivy image + fs/secret scan
      → gate on HIGH/CRITICAL → ECR (IMMUTABLE tags, scan_on_push) → digest recorded
      → GitOps commit → ArgoCD → Gatekeeper (approved registry, no :latest) → PSA
```

- Images are multi-stage and non-root (UID 10001) and contain no shell tooling at runtime. See `docker/api-service/Dockerfile`.
- GKE has **Binary Authorization** enabled, so only attested images run on the data plane.
- ECR lifecycle policy expires untagged images, which limits the attack surface of stale images.

---

6. Data protection

Data	At rest	In transit	Access
RDS MySQL	KMS CMK (`storage_encrypted`), Performance Insights encrypted with the same CMK	`require_secure_transport = ON`	SG from EKS nodes only, IAM DB auth
EKS secrets	KMS envelope encryption (`encryption_config`)	TLS	RBAC
S3 backups / audit	SSE-KMS, bucket key, versioning, public access block, `BucketOwnerEnforced`	Bucket policy **denies** `aws:SecureTransport = false`	IRSA prefix-scoped (`api-service/*`)
CloudTrail	SSE-KMS, log-file validation (tamper evidence)	TLS	Audit bucket, 6-year retention
Azure Key Vault	HSM-backed keys, soft delete 90 d, **purge protection**	TLS 1.2+	Access policies per identity
GKE secrets / GCS / BigQuery	Cloud KMS CMEK, rotated every 90 days	TLS	Workload Identity, dataset-scoped IAM

---

7. Detection and response

Signal	Source	Route
Threats (crypto-mining, credential exfiltration, anomalous API calls, EKS runtime)	GuardDuty with S3, EKS audit, RDS login, EBS malware and **Runtime Monitoring** features	EventBridge (severity ≥ 7) → SNS `security-alerts` (KMS)
API activity	CloudTrail (multi-region, ApiCallRateInsight)	S3 audit bucket
Azure control plane	Diagnostic settings on AKS, Key Vault and ACR	Log Analytics
AKS runtime	Microsoft Defender for Containers	Defender for Cloud
Service health / SLO	Prometheus rules (`monitoring/prometheus/custom-rules/api-alerts.yml`)	Alertmanager → on-call
Security posture	`scripts/compliance_scan.py all` (nightly) and the Security Posture Grafana dashboard	CI artefact, dashboard
Host integrity	auditd rules (`ansible/roles/cis_hardening/templates/audit.rules`)	Fluent Bit → Loki